

EXERCISE 7

INTRODUCTION TO PUBLIC-KEY CRYPTOGRAPHY

1. PRACTICAL SESSION

1. Consider the Extended Euclidean Algorithm (recursive form) in Python as follows:

```
1 def extended_euclid(a, b):
2     if b == 0:
3         return a, 1, 0
4     else:
5         d_prime, x_prime, y_prime = extended_euclid(b, a % b)
6         d = d_prime
7         x = y_prime
8         y = x_prime - (a // b) * y_prime
9         return d, x, y
```

Modify the above algorithm so that it outputs a table similar to the following:

a	b	$\lfloor a/b \rfloor$	d	x	y
5	12	0	1	5	-2
12	5	2	1	-2	5
5	2	2	1	1	-2
2	1	2	1	0	1
1	0	-	1	1	0

2. Consider the algorithm for finding the modular inverse based on the extended Euclidean algorithm.

```
1 def mod_inv(a, n):
2     d, x, y = extended_euclid(a, n)
3     if d != 1:
4         raise ValueError(f"The modular inverse of {a} modulo {n}
5                             does not exist.")
6     else:
7         return x % n
```

Implement and test the modular inverse calculation for a 128-bit integer.

Given:

- Let $a = 340282366920938463463374607431768211457$ (which is $2^{128} + 1$)
- Let $n = 6277101735386680763835789423207666416102355444464034512896$ (a large 128-bit number, near 2^{192})

Task: Find the integer b such that:

$$a \cdot b \equiv 1 \pmod{n}$$

To solve this, use the function `mod_inv(a, n)`.

Date: October 30, 2024.